Lab sheet 5

ANISH TIWARI

section→c

roll no.→11

Experiment Name

Feature Extraction and Image Analysis using SIFT and HOG Descriptors

Aim

To implement Scale-Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients(HOG) techniques for feature extraction and analyze their effectiveness in imagerepresentation, object recognition, and computer vision applications.

Step 1: Import Required Libraries

In [3]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.feature import hog
from skimage import exposure
```

Step 2: Load and Preprocess Image

In [6]:
```
filename = "my_pic01.png"
img = cv2.imread(filename)
```

In [8]:
```
filename = "my_pic01.png"
img = cv2.imread(filename)

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

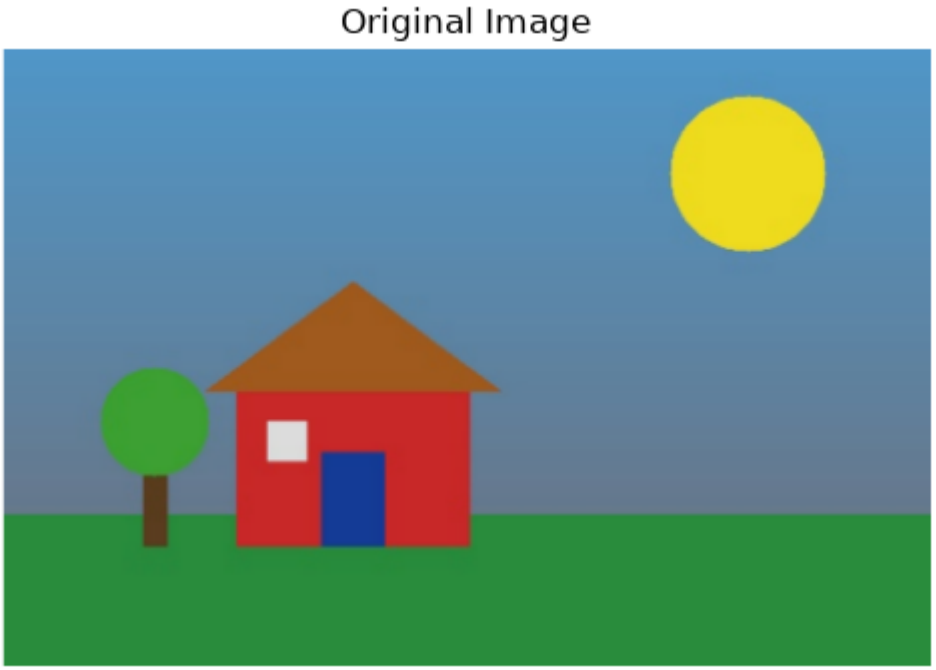
print("Image loaded successfully!")
print("Image shape:", img.shape)
```

```
Image loaded successfully!
Image shape: (303, 458, 3)
```

In [9]:
```
plt.figure(figsize=(6, 4))

plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.show()
```



Step 3: Apply SIFT

In [ ]:
```
# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and compute descriptors
keypoints, descriptors = sift.detectAndCompute(gray, None)

print("Number of keypoints:", len(keypoints))
print("Descriptor shape:", descriptors.shape)
```

```
Number of keypoints: 26
Descriptor shape: (26, 128)
```

In [16]:
```
# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints
keypoints, descriptors = sift.detectAndCompute(gray, None)

print("Number of keypoints:", len(keypoints))
```

```
Number of keypoints: 26
```

Step 4: Visualize SIFT Keypoints

In [17]:
```
# Draw SIFT keypoints on image

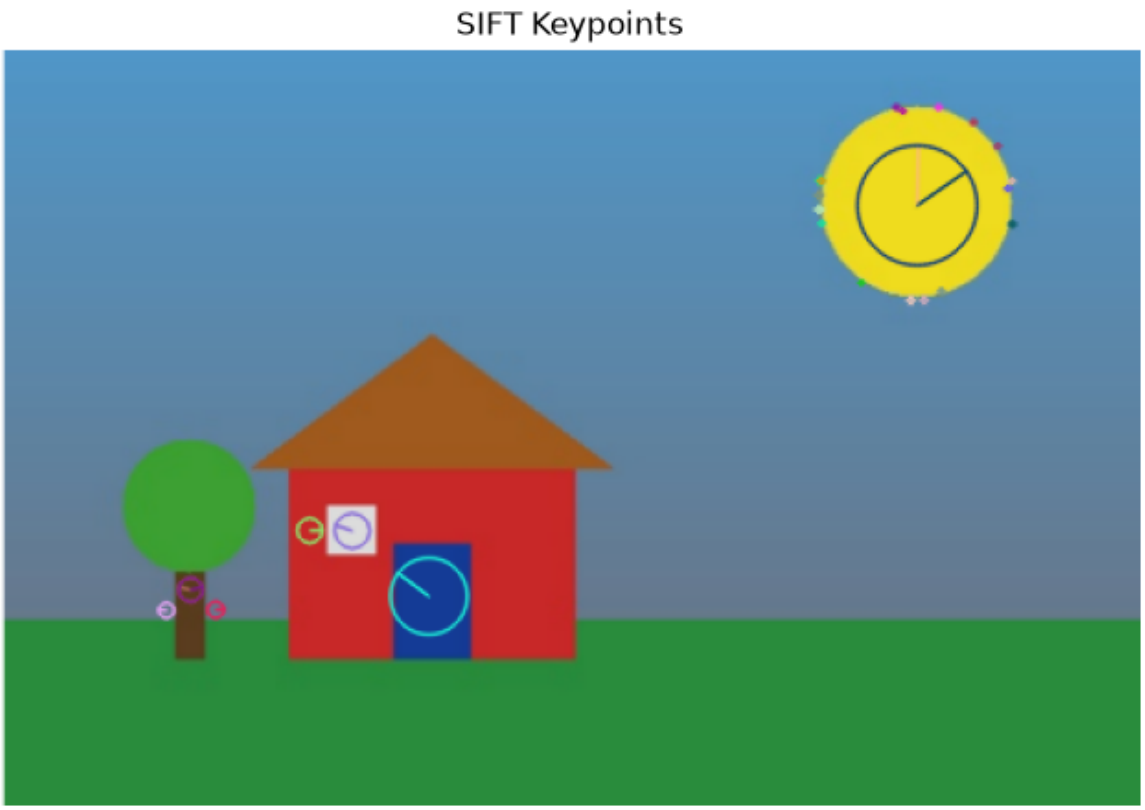
sift_image = cv2.drawKeypoints(
    img_rgb,
    keypoints,
```

```
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(8, 6))

plt.imshow(sift_image)
plt.title("SIFT Keypoints")
plt.axis("off")

plt.show()
```



Step 5: Extract HOG Features

In [12]:
```
# HOG parameters
orientations = 9
pixels_per_cell = (8, 8)
cells_per_block = (2, 2)

# Extract HOG features
hog_features, hog_image = hog(
    gray,
    orientations=orientations,
    pixels_per_cell=pixels_per_cell,
    cells_per_block=cells_per_block,
    block_norm='L2-Hys',
    visualize=True
)

print("HOG feature length:", len(hog_features))
```

```
HOG feature length: 72576
```

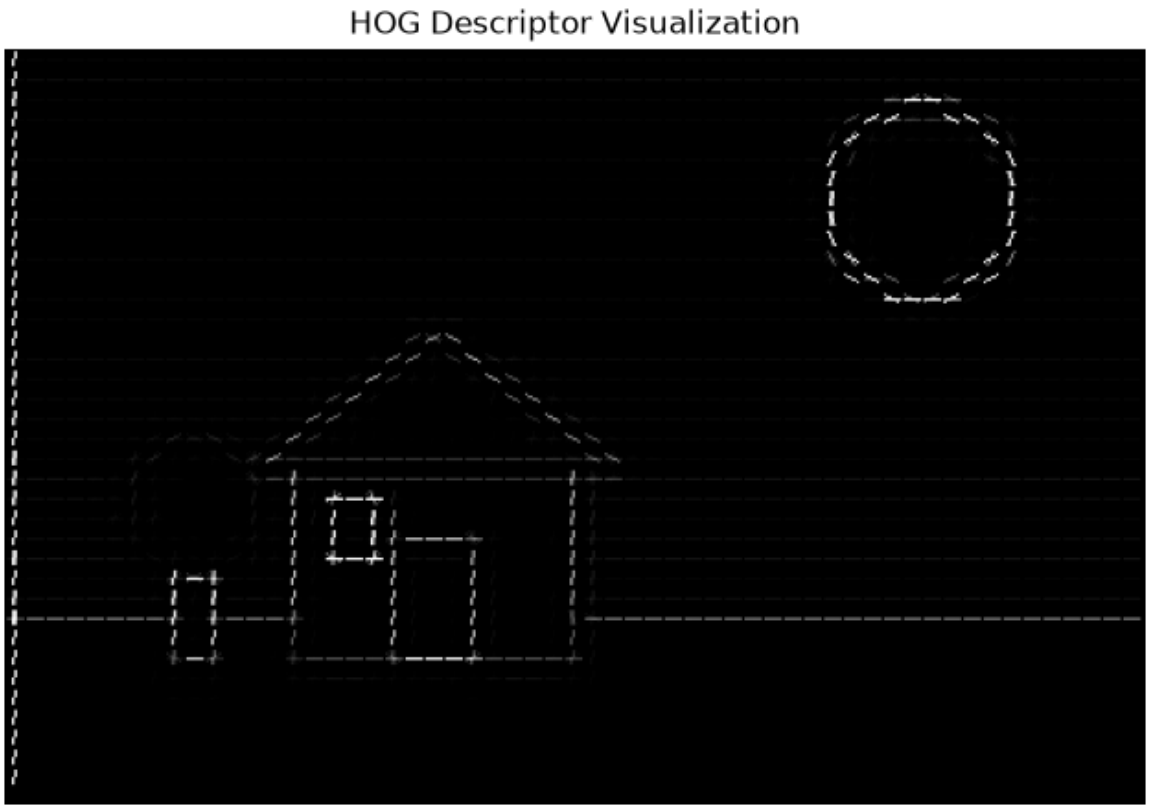
Step 6: Visualize HOG

In [13]:
```
# Improve HOG visualization
hog_image_rescaled = exposure.rescale_intensity(
    hog_image,
    in_range=(0, 10)
)

plt.figure(figsize=(8, 6))

plt.imshow(hog_image_rescaled, cmap='gray')
plt.title("HOG Descriptor Visualization")
plt.axis("off")

plt.show()
```



Step 7: Compare SIFT and HOG

In [18]:
```
print("========== SIFT ==========")
print("Number of Keypoints:", len(keypoints))
print("Descriptor Dimension:", descriptors.shape[1])

print("\n========== HOG ==========")
print("Number of HOG Features:", len(hog_features))
print("Orientations:", orientations)
print("Pixels per Cell:", pixels_per_cell)
print("Cells per Block:", cells_per_block)
```

```
========== SIFT ==========
Number of Keypoints: 26
Descriptor Dimension: 128

========== HOG ==========
Number of HOG Features: 72576
Orientations: 9
Pixels per Cell: (8, 8)
Cells per Block: (2, 2)
```

In [21]:
```python
# Create second similar image

img2 = cv2.resize(img_rgb, None, fx=0.8, fy=0.8)

gray2 = cv2.cvtColor(img2, cv2.COLOR_RGB2GRAY)

sift2 = cv2.SIFT_create()

keypoints2, descriptors2 = sift2.detectAndCompute(gray2, None)

print("Image 1 keypoints:", len(keypoints))
print("Image 2 keypoints:", len(keypoints2))
```

```
Image 1 keypoints: 26
Image 2 keypoints: 17
```

In [ ]:
```python
# Match SIFT descriptors

bf = cv2.BFMatcher()

matches = bf.knnMatch(descriptors, descriptors2, k=2)

good_matches = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Total matches:", len(matches))
print("Good matches:", len(good_matches))
```

Step 8: Image Matching using SIFT

In [22]:
```python
# Create a second similar image
img2 = cv2.resize(
    img_rgb,
    None,
    fx=0.8,
    fy=0.8
)

gray2 = cv2.cvtColor(img2, cv2.COLOR_RGB2GRAY)

# Create SIFT detector
sift2 = cv2.SIFT_create()

# Detect keypoints and descriptors
keypoints2, descriptors2 = sift2.detectAndCompute(gray2, None)

print("Image 1 keypoints:", len(keypoints))
```

```
print("Image 2 keypoints:", len(keypoints2))
```

```
Image 1 keypoints: 26
Image 2 keypoints: 17
```

In [23]:
```
# Create BFMatcher
bf = cv2.BFMatcher()

# KNN matching
matches = bf.knnMatch(descriptors, descriptors2, k=2)

# Apply Lowe's ratio test
good_matches = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Total matches:", len(matches))
print("Good matches:", len(good_matches))
```

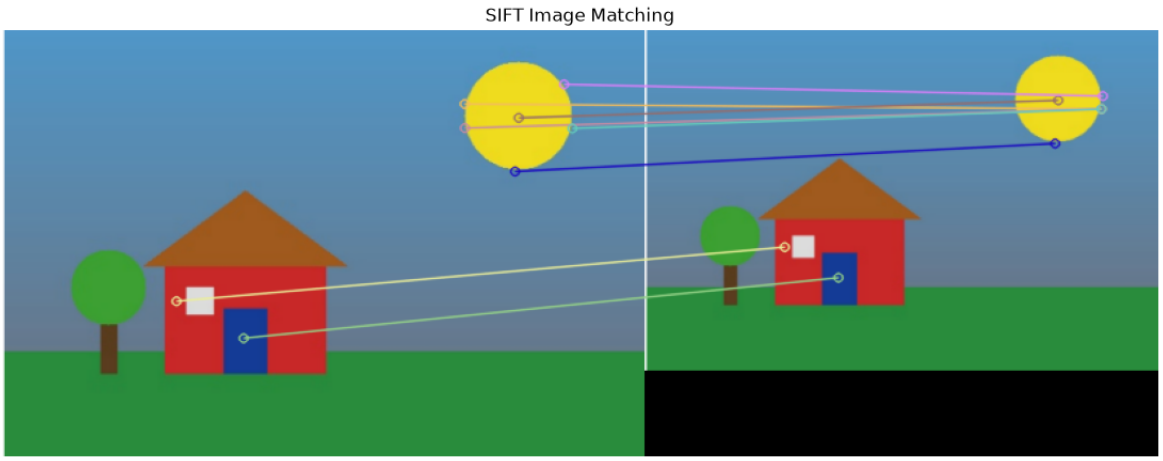
```
Total matches: 26
Good matches: 8
```

In [24]:
```
# Draw good matches
match_image = cv2.drawMatches(
    img_rgb,
    keypoints,
    img2,
    keypoints2,
    good_matches[:50],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(14, 7))

plt.imshow(match_image)
plt.title("SIFT Image Matching")
plt.axis("off")

plt.show()
```



Step 9: Evaluate SIFT and HOG

In [25]:
```
print("========== EVALUATION ==========")

print("\nSIFT:")
print("- Detects local keypoints")
print("- Provides 128-dimensional descriptors")
print("- Robust to scale and rotation changes")
print("- Useful for image matching and object recognition")

print("\nHOG:")
print("- Represents image using gradient orientations")
print("- Captures shape and edge information")
print("- Computationally simpler than SIFT")
print("- Useful for object detection and shape analysis")
```

```
========== EVALUATION ==========

SIFT:
- Detects local keypoints
- Provides 128-dimensional descriptors
- Robust to scale and rotation changes
- Useful for image matching and object recognition

HOG:
- Represents image using gradient orientations
- Captures shape and edge information
- Computationally simpler than SIFT
- Useful for object detection and shape analysis
```

Step 10: Final Visualization

In [26]:
```
plt.figure(figsize=(14, 10))

# Original
plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

# Grayscale
plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

# SIFT
plt.subplot(2, 3, 3)
plt.imshow(sift_image)
plt.title("SIFT Keypoints")
plt.axis("off")

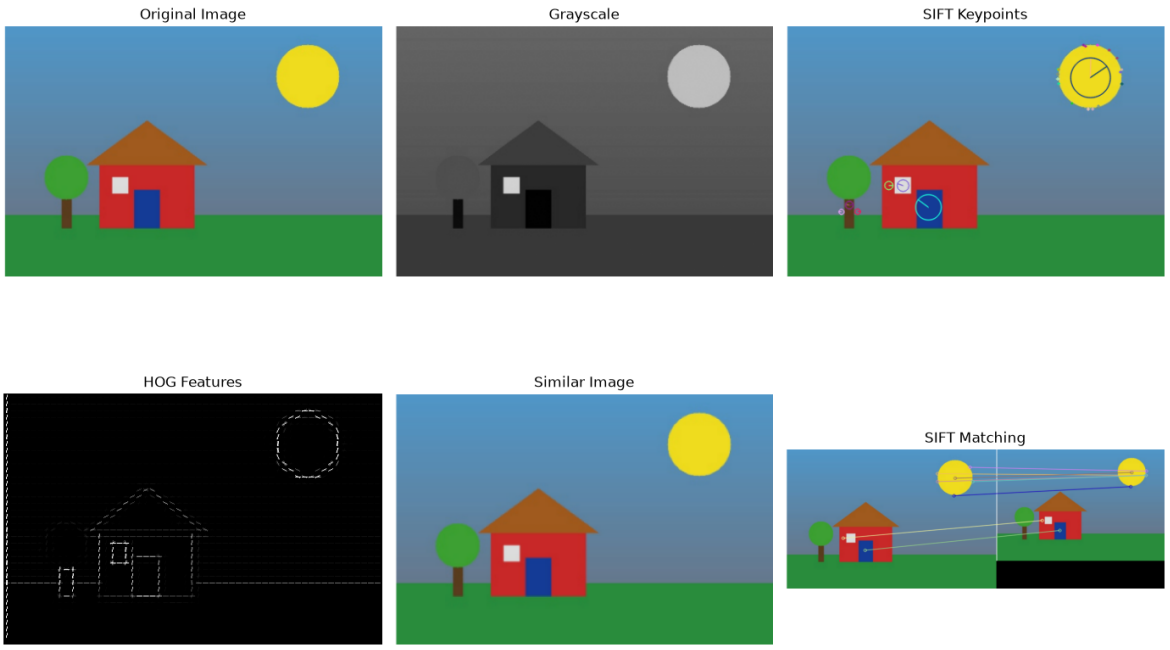
# HOG
plt.subplot(2, 3, 4)
plt.imshow(hog_image_rescaled, cmap="gray")
plt.title("HOG Features")
plt.axis("off")

# Second image
plt.subplot(2, 3, 5)
plt.imshow(img2)
```

```
plt.title("Similar Image")
plt.axis("off")

# Matches
plt.subplot(2, 3, 6)
plt.imshow(match_image)
plt.title("SIFT Matching")
plt.axis("off")

plt.tight_layout()
plt.show()
```



1. What is feature extraction, and why is it important?

   Feature extraction finds important patterns or characteristics from an image. It helps in image matching, recognition, tracking, and classification.

2. Explain the working principle of SIFT.

   SIFT detects important keypoints in an image and generates descriptors for them. It is designed to handle changes in scale and rotation.

3. What are keypoints and feature descriptors?

   Keypoints are important locations in an image, such as corners or distinctive regions. Descriptors are numerical representations of those keypoints used for comparison.

4. Explain HOG and its significance.

   HOG (Histogram of Oriented Gradients) represents an image using gradient directions and their strengths. It is useful for capturing shape and edge information.

5. Compare SIFT and HOG. SIFT: More robust to scale and rotation; useful for image matching. HOG: Focuses on shape and edges; generally simpler and

useful for object detection.

6. Why is SIFT invariant to scale and rotation?

   SIFT detects features at different scales and assigns an orientation to keypoints, allowing the same feature to be recognized after resizing or rotation.

7. Three applications of HOG. Pedestrian detection Object detection Human shape/gesture analysis

8. Why perform feature extraction before classification?

   It reduces raw image data into meaningful features, making classification and object detection easier and more efficient.

9. Handcrafted vs deep-learning features.

   Advantages: Simple, interpretable, and can work with smaller datasets. Limitations: Less adaptable and often less powerful than deep-learning features for complex tasks.

10. How do feature extraction techniques help?

    They provide useful image features for matching similar images, recognizing faces, and detecting objects.